

DurielMedic Payment Architecture

Production-ready Paystack design and Codex implementation guide for Django

Goal: replace a callback/session-heavy payment flow with a persistent, verifiable, idempotent payment system that can safely handle new clinic registrations and subscription renewals.

What this guide changes

- Create a local payment transaction before redirecting the customer to Paystack.
- Let the backend - not the browser - decide the price and whether payment succeeded.
- Support both Paystack callback and webhook, while routing both through one verification/fulfilment service.
- Verify status, reference, amount and currency before activating a subscription.
- Make payment processing idempotent so refreshes or duplicate webhooks cannot activate twice.
- Persist pending registrations instead of relying primarily on Django session state.
- Keep payment confirmation and subscription fulfilment as separate responsibilities.

1. Target architecture

```
User chooses plan
|
Django resolves trusted server-side price
|
Create PendingClinicRegistration
|
Create PaymentTransaction (PENDING)
|
Initialize Paystack with local reference
|
Paystack checkout
|
+-----+
| |
Browser callback Paystack webhook
| |
+-----+-----+
|
confirm_payment(reference)
|
Verify with Paystack API
|
status/reference/amount/currency valid?
|
YES
|
transaction.atomic()
|
already PAID?
/ \
yes no
| |
finish mark PAID
|
fulfil payment
|
create/activate/renew clinic
|
commit
|
send email/notification
```

The callback improves the customer's browser experience. The webhook gives the server a second, independent way to learn about successful payments. Neither should contain separate business logic; both call the same service.

2. Recommended data model

Create a dedicated payments app if the project does not already have one. The exact field names can be adapted to the existing models.

```
# payments/models.py
class PaymentTransaction(models.Model):
class Status(models.TextChoices):
    PENDING = "PENDING"
    PROCESSING = "PROCESSING"
    PAID = "PAID"
    FAILED = "FAILED"
    CANCELLED = "CANCELLED"
    REFUNDED = "REFUNDED"

reference = models.CharField(max_length=100, unique=True)
clinic = models.ForeignKey(
    Clinic, null=True, blank=True,
    on_delete=models.SET_NULL,
    related_name="payment_transactions",
```

```

)
email = models.EmailField()
plan_type = models.CharField(max_length=20)
amount = models.DecimalField(max_digits=12, decimal_places=2)
currency = models.CharField(max_length=3, default="NGN")
provider = models.CharField(max_length=30, default="PAYSTACK")
status = models.CharField(
    max_length=20,
    choices=Status.choices,
    default=Status.PENDING,
)
provider_response = models.JSONField(default=dict, blank=True)
paid_at = models.DateTimeField(null=True, blank=True)
created_at = models.DateTimeField(auto_now_add=True)
updated_at = models.DateTimeField(auto_now=True)

class PendingClinicRegistration(models.Model):
    payment = models.OneToOneField(
        PaymentTransaction,
        on_delete=models.CASCADE,
        related_name="pending_registration",
    )
    clinic_name = models.CharField(max_length=255)
    clinic_type = models.CharField(max_length=30)
    clinic_address = models.TextField(blank=True)
    clinic_phone = models.CharField(max_length=50, blank=True)
    clinic_email = models.EmailField(blank=True)
    username = models.CharField(max_length=150)
    email = models.EmailField()
    # Store password safely according to the chosen registration design.
    # Do NOT persist plaintext passwords.
    registration_payload = models.JSONField(default=dict, blank=True)
    expires_at = models.DateTimeField()
    completed_at = models.DateTimeField(null=True, blank=True)

```

Important: do not put a plaintext password in `registration_payload`. Prefer creating an inactive user with Django's normal password hashing, or store only the minimum pending data needed and use a secure account-setup step after payment.

3. Trusted pricing

The client may request a plan, but the backend determines its price. Never activate a plan using an amount supplied by a form or JavaScript.

```

PLAN_PRICES = {
    "MONTHLY": Decimal("15000.00"),
    "YEARLY": Decimal("150000.00"),
}

def get_plan_amount(plan_type):
    try:
        return PLAN_PRICES[plan_type]
    except KeyError:
        raise ValidationError("Invalid subscription plan.")

```

4. Payment initialization service

```

def initialize_payment(*, email, plan_type, clinic=None):
    amount = get_plan_amount(plan_type)
    reference = f"DM-{uuid.uuid4().hex.upper()}"

    payment = PaymentTransaction.objects.create(
        reference=reference,
        clinic=clinic,
        email=email,
        plan_type=plan_type,
        amount=amount,
    )

```

```

currency="NGN",
)

payload = {
    "email": email,
    "amount": int(amount * 100), # Paystack uses kobo
    "reference": reference,
    "callback_url": build_callback_url(),
}

# POST payload to Paystack /transaction/initialize
# Save useful non-sensitive provider response.
# Return authorization_url to the view.
return payment, authorization_url

```

The local reference is created first. That gives DurielMedic a durable record even if the user closes the page, loses internet or never reaches the callback.

5. One verification service

This is the heart of the design. Callback and webhook both call this service.

```

def confirm_paystack_payment(reference):
    # 1. Fetch local PaymentTransaction.
    # 2. Ask Paystack /transaction/verify/{reference}.
    # 3. Validate:
    # provider status == success
    # provider reference == local reference
    # provider amount == local amount * 100
    # provider currency == local currency
    # 4. Enter transaction.atomic().
    # 5. Re-fetch with select_for_update().
    # 6. If already PAID, return safely.
    # 7. Mark PAID and record paid_at/provider response.
    # 8. Fulfil the purchase exactly once.
    # 9. Commit.
    # 10. Trigger non-critical notifications after commit.

```

A successful Paystack status alone is not sufficient. The successful transaction must be the exact transaction DurielMedic expected.

6. Idempotency and database locking

Callbacks can be refreshed and webhooks can be delivered more than once. The same payment must therefore be safe to process repeatedly.

```

with transaction.atomic():
    payment = (
        PaymentTransaction.objects
        .select_for_update()
        .get(reference=reference)
    )

    if payment.status == PaymentTransaction.Status.PAID:
        return payment

    # validated Paystack result
    payment.status = PaymentTransaction.Status.PAID
    payment.paid_at = timezone.now()
    payment.provider_response = verified_data
    payment.save()

    fulfil_payment(payment)

```

`select_for_update()` locks that transaction row while it is being processed. The second request waits, then sees that the payment is already PAID and does not extend the subscription again.

7. Fulfilment service

```
def fulfil_payment(payment):
    if payment.clinic_id:
        # Existing clinic renewal
        payment.clinic.set_subscription(payment.plan_type)
        return

    # New registration
    pending = payment.pending_registration

    # Create clinic and admin using the pending registration.
    # Activate the selected plan.
    # Mark pending registration completed.
    # This function must only run after verified payment.
```

Payment verification answers *'did the expected money arrive?'*. Fulfilment answers *'what should DurielMedic do because it arrived?'*. Keeping these separate makes the code much easier to test.

8. Callback

```
def paystack_callback(request):
    reference = request.GET.get("reference")
    if not reference:
        return redirect("payment_failed")

    try:
        payment = confirm_paystack_payment(reference)
    except PaymentVerificationError:
        return redirect("payment_failed")

    return redirect("payment_success")
```

The callback is no longer the only source of truth. It simply invokes the common confirmation service and shows the correct user-facing page.

9. Webhook

```
@csrf_exempt
@require_POST
def paystack_webhook(request):
    # Validate Paystack webhook authenticity/signature first.
    # Parse event.
    # Ignore unrelated event types.
    # Extract reference from charge.success.
    # Call confirm_paystack_payment(reference).
    # Return HTTP 200 after successful handling.

    return HttpResponse(status=200)
```

The implementation must validate Paystack's webhook signature according to the current Paystack documentation before trusting the event. Do not treat an arbitrary POST to this endpoint as proof of payment.

10. New registration flow

```
Registration form
|
Validate form
|
Create durable pending registration
|
Create PENDING payment
|
Redirect to Paystack
|
Verified payment
```

```
|
Atomic fulfilment
|
Create Clinic
|
Create ADMIN user
|
clinic.set_subscription(plan_type)
|
Mark pending registration complete
```

This removes the payment's dependence on the browser session. Session data may still be used for convenience, but it is no longer the durable record of the purchase.

11. Renewal flow

```
Authenticated clinic admin
|
Select MONTHLY/YEARLY
|
Server resolves amount
|
PaymentTransaction(clinic=current_clinic, PENDING)
|
Paystack
|
Verified payment
|
Atomic fulfilment
|
clinic.set_subscription(plan_type)
|
Payment = PAID
```

12. Failure handling

- Paystack initialization fails: keep or mark the transaction appropriately and show a retryable error.
- Verification returns failed: do not activate anything; record the provider response and set FAILED only when appropriate.
- Verification API is temporarily unavailable: do not mark the payment failed merely because verification could not be completed. Leave it recoverable/pending and retry.
- Email fails: payment and subscription activation remain committed. Email is non-critical.
- Duplicate callback/webhook: return the already-paid transaction without applying the subscription twice.
- Amount/currency/reference mismatch: reject fulfilment and log the mismatch for investigation.

13. Admin and reconciliation

Expose PaymentTransaction in Django admin with filters for status, provider, plan and date. Search by reference and email. This gives support staff a reliable answer when a clinic says it paid but was not activated.

```
Reference Clinic Plan Amount Status
DM-A72FD2 Grace Clinic MONTHLY 15,000 PAID
DM-81BC09 Vision Eye YEARLY 150,000 PAID
DM-F30A91 Hope Clinic MONTHLY 15,000 FAILED
DM-C892AF Unity Clinic MONTHLY 15,000 PENDING
```

14. Tests Codex should add

- Successful new clinic payment creates one clinic, one admin and one subscription activation.

- Successful renewal extends the correct clinic exactly once.
- Failed Paystack verification creates no clinic and no active subscription.
- Amount mismatch is rejected.
- Currency mismatch is rejected.
- Unknown reference is rejected.
- Calling confirmation twice does not fulfil twice.
- Callback followed by webhook does not fulfil twice.
- Webhook followed by callback does not fulfil twice.
- Temporary verification/network failure leaves payment recoverable.
- Email failure does not roll back a successful payment.
- Trial registration bypasses Paystack but remains separate from paid fulfilment.

15. Suggested implementation order for Codex

1	Inspect existing code	Find Clinic.set_subscription(), registration/renewal views, URLs, settings, user model and any existing payment m
2	Add models	Create PaymentTransaction and a safe durable pending-registration design. Add migrations and Django admin.
3	Extract pricing	Create one trusted server-side plan pricing function and update callers.
4	Add services	Implement initialize_payment(), verify/confirm payment and fulfil_payment().
5	Refactor initialization	Create local transaction before calling Paystack and send its reference to Paystack.
6	Refactor callback	Make callback call the common confirmation service.
7	Add webhook	Validate webhook authenticity, extract reference and call the same confirmation service.
8	Add idempotency	Use transaction.atomic(), select_for_update(), unique reference and PAID short-circuit.
9	Add tests	Cover success, failure, mismatch, duplicate delivery and renewal/new-registration paths.
10	Deploy safely	Run migrations, configure webhook URL/secret requirements, test with Paystack test mode, then switch production

16. Prompt to give Codex in VS Code

Give Codex the relevant project files first, then use a prompt along these lines:

```
Review the existing DurielMedic Django Paystack implementation before editing it.

Goal:
Refactor payments into a production-ready design without breaking existing clinic registration, TRIAL
registration, or subscription renewal.

Requirements:
1. Add a persistent PaymentTransaction model with a unique local reference, expected amount,
currency, plan, clinic where applicable, email, provider status/response, paid_at and timestamps.
2. Replace payment-critical session dependence with a durable pending-registration mechanism. Never
store plaintext passwords.
3. The backend must determine plan prices.
4. Create the local transaction BEFORE initializing Paystack and pass the local reference to
Paystack.
5. Implement a reusable payment confirmation service.
6. Verify Paystack status, reference, amount and currency before fulfilment.
7. Make fulfilment idempotent using transaction.atomic(), select_for_update() and an already-PAID
guard.
8. Use the same confirmation service from both the browser callback and Paystack webhook.
9. Validate webhook authenticity according to Paystack's current documented signature mechanism.
10. Separate verification from fulfilment.
11. New registration: create/activate the clinic and admin only after confirmed paid transaction.
12. Renewal: renew the existing clinic only after confirmed paid transaction.
13. Preserve the existing TRIAL flow but keep it separate from paid payment verification.
14. Email/notifications must not be able to roll back a successful payment.
15. Add Django admin support and tests for success, failure, amount mismatch, currency mismatch,
unknown reference, duplicate callback, duplicate webhook, callback+webhook, renewal and new
registration.

Work incrementally:
- First inspect and explain the current payment-related files and models.
- Then propose exact file changes.
- Implement in small coherent steps.
- Generate migrations.
- Run Django checks and relevant tests after each major step.
- Do not delete existing working behavior unless the replacement is implemented and tested.
- Flag any assumptions before making schema decisions.
```

17. Files to give Codex

- The current views.py containing register_facility(), paystack_payment() and paystack_callback().
- Clinic and CustomUser models, especially Clinic.set_subscription().
- FacilityRegistrationForm and any plan/renewal forms.
- Payment/registration URL configuration.
- settings.py sections for Paystack, email and installed apps - redact real secret values if sharing outside your local environment.
- Existing payment models, if any.
- Relevant registration, payment success/failure and renewal templates.
- Existing tests for registration/subscription/payment.

Do not paste production secret keys into prompts, screenshots, commits or test fixtures. Codex working locally can refer to environment-variable names without needing the real secret in generated code.

18. Definition of done

- A payment attempt exists in the database before Paystack checkout.

- A successful browser redirect alone cannot activate a subscription.
- A forged webhook cannot activate a subscription.
- A successful payment with the wrong amount cannot activate a subscription.
- A payment can be processed repeatedly without duplicate fulfilment.
- Successful payment still completes if the browser never returns, provided the valid webhook is received.
- Support can locate transactions by reference/email/status.
- Tests prove new registration and renewal behave correctly.
- TRIAL continues to work without Paystack.
- No plaintext password is persisted in pending-payment data.

Implementation note

This document is an implementation blueprint based on the payment flow supplied for DurielMedic. Before coding the webhook signature verification and provider-specific response fields, Codex should check the current Paystack documentation and match the installed project's Django/Python versions.